

1. High-Level Architecture

Streamlit UI	FastAPI API	Gemini generator	Export services
Form input; preview; edit; downloads	Validation; /health; /generate	Prompt; model discovery; retry; fallback; demo mode	TXT; DOCX; PDF formatting

Flow: User enters facts -> Streamlit POST request -> FastAPI validation -> GeminiDocumentGenerator -> generated Markdown draft -> response to Streamlit -> preview/edit -> export service.

2. Module Design

Module	Responsibility	Key files
Frontend	UI, state, form validation, preview and download controls	frontend/app.py
API	HTTP routes and response handling	backend/main.py, backend/routes.py
Schema	Request/response models and input validation	backend/schemas.py
AI core	Prompt construction, model discovery, retry and fallback	backend/ai_core/gemini_generator.py
Text utilities	Sanitization, Markdown parsing, preview HTML, filenames	backend/utis/text.py
Export services	Convert a draft into TXT, DOCX and PDF bytes	backend/services/*formatter.py, exporters.py
Tests	Automated regression tests	tests/test_api.py, tests/test_exporters.py, tests/test_text.py

3. Data Flow

Step	Action
1	User chooses document type and enters parties, terms and dates.
2	Streamlit checks for missing inputs before the request is sent.
3	FastAPI validates the JSON payload using Pydantic.
4	Generator sanitizes content and builds a prompt that forbids invented material facts.
5	Accessible configured Gemini models are considered; temporary provider failures are retried.
6	The generated Markdown document is returned as a response model.
7	Streamlit renders a safe preview and enables editing/export.

4. Error and Safety Design

- HTTP 422 is expected for schema validation failures.
- Provider or generation failures are returned as HTTP 502 details for UI display.
- The preview uses escaped HTML rather than injecting raw user-generated HTML.
- The prompt explicitly instructs the model not to invent material facts and to use placeholders instead.
- The sidebar includes a human-review warning because the output is an AI-assisted draft.

5. Repository Design

LegalEase/
 backend/
 frontend/
 tests/
 assets/
requirements.txt
Dockerfile / Procfile
run scripts + configuration